JWT 발급 과제

JSONPlaceholder 사용자 조회부터 10분 토큰 발급, 실제 화면 검증까지

양유상

멋쟁이사자처럼 14기 | 실행·검증: 2026.10.24 (KST)

01 구현 결과

사용자명 **Bret**으로 외부 API의 사용자를 찾고, 사용자 정보 **4개**를 포함한 JWT를 발급했다. 서명 알고리즘은 **HS256**, 유효시간은 **10분(600초)**으로 설정했다.

정상 요청은 HTTP 200, 없는 사용자는 404, 잘못된 입력은 400으로 처리했다. 자동 테스트 8개와 실제 서버 요청 5개를 검증했다.

02 과제 GitHub 주소

구현 저장소
https://github.com/yangyu0330/likelion-jwt-assignment

제공된 기본 코드
https://github.com/tjwls11/likelion-jwt

03 구현 흐름

1. 기본 코드 clone 및 npm install 실행
2. 무작위 32바이트 키를 생성하고 .env에 JWT_SECRET 설정
3. fetchUsers()로 외부 사용자 목록 조회
4. username 정확 일치 검색 후 Payload 구성 및 jwt.sign() 호출
5. 실제 POST /login 요청, jwt.io Payload 확인, 서명·만료 테스트

화면 자료는 실행 중인 브라우저를 직접 캡처했다. 생성 이미지나 재현한 가짜 화면은 사용하지 않았다.

구현 과정과 핵심 코드

기본 코드의 함수 구조를 유지하면서 사용자 검색·Payload·서명을 완성했다.

01 환경 설정

```
npm install
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
npm run dev
```

생성된 64자리 문자열은 프로젝트 최상위 .env의 JWT_SECRET에 저장했다. PORT는 3000으로 설정했고, .env는 .gitignore로 제외했다. 실제 키는 보고서와 저장소에 포함하지 않았다.

02 src/auth.js 핵심 구현

```
const users = await fetchUsers(fetcher)
const user = users.find((user) => user.username === username)

if (!user) {
  const error = new Error('사용자 없음')
  error.status = 404
  throw error
}

const payload = {
  sub: String(user.id),
  username: user.username,
  name: user.name,
  email: user.email,
}

const token = jwt.sign(payload, secret, {
  algorithm: 'HS256',
  expiresIn: '10m',
})
```

issueToken의 secret 기본값은 process.env.JWT_SECRET이다. 기본 코드 README의 조건에 따라 사용자 id는 문자열 sub로 저장하고, phone과 address는 제외했다.

추가 처리: username의 문자열·빈 값 검증, 외부 API 10초 제한, 외부 API 실패 502, 본문 누락 및 잘못된 JSON의 400 응답.

실제 로그인 요청과 응답

실습용 브라우저 UI에서 실행 중인 서버로 POST /login을 전송했다.

요청: POST http://127.0.0.1:3000/login
본문: {"username":"Bret"}
결과: HTTP 200 OK · user 정보와 token 반환

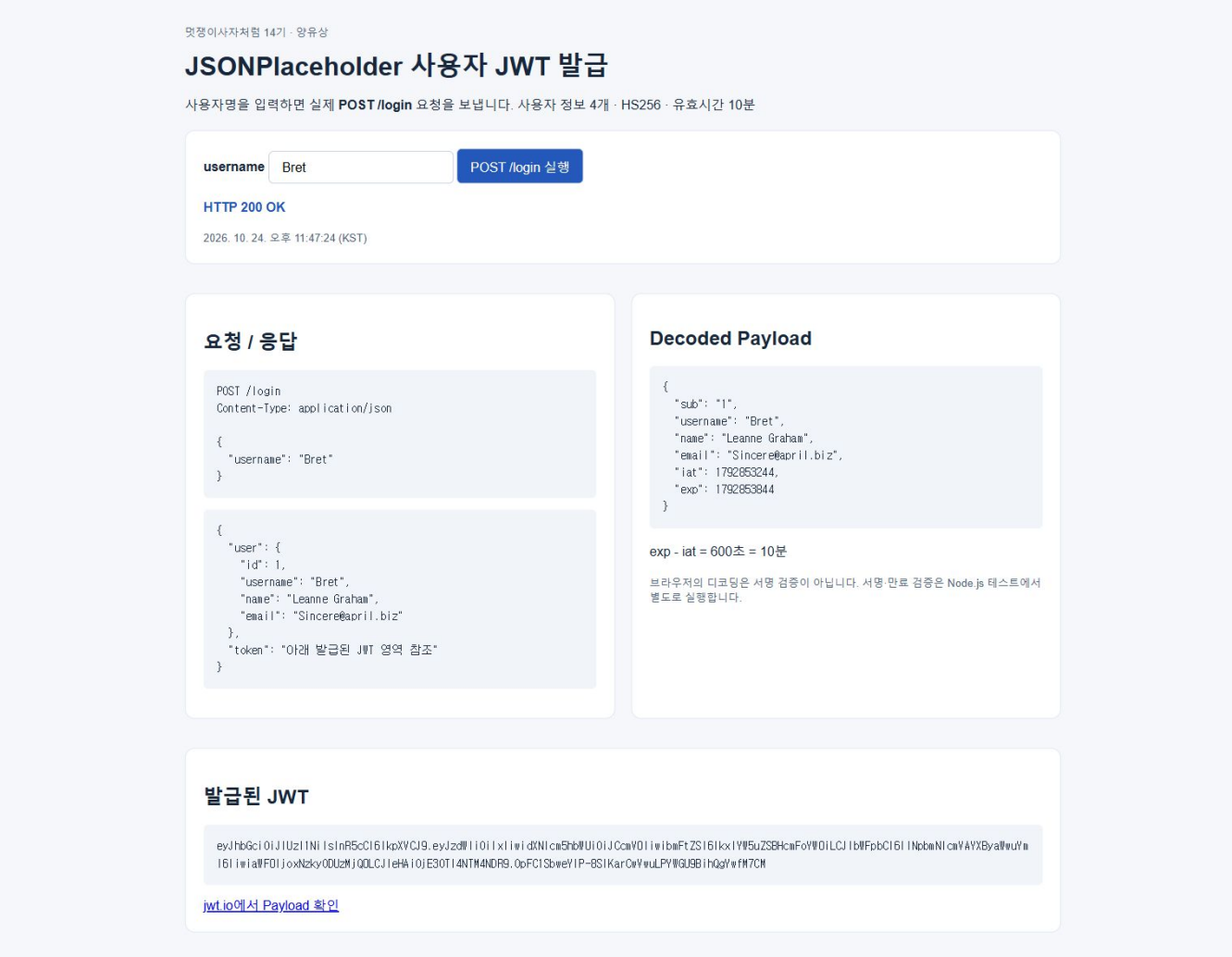
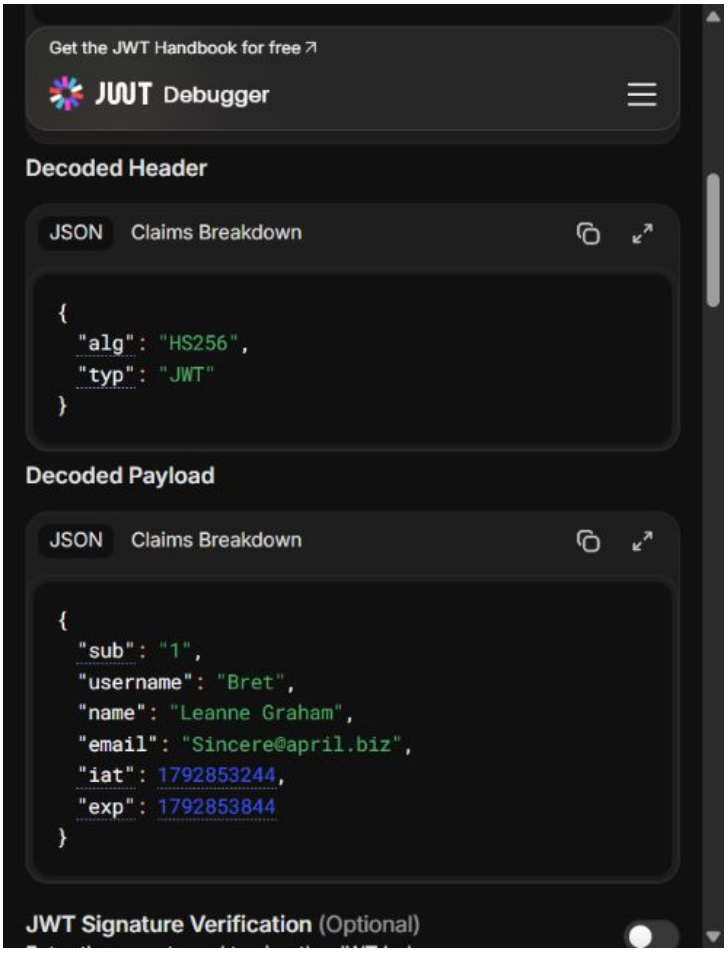


그림 1. 직접 캡처한 로그인 성공 화면 · 2026.10.24 23:47:24 KST

이 페이지는 제출 검증용으로 추가한 실제 요청 UI이다. 응답 영역의 token 안내 문구는 긴 토큰을 아래 영역에 별도로 표시하기 위한 UI 처리이며, 실제 API 응답에는 전체 token 문자열이 포함된다.

jwt.io에서 Payload 확인

로그인 성공 화면의 토큰을 그대로 jwt.io 디코더에 입력했다.



사용자 정보 4개

sub

사용자 id: "1"

username

Bret

name

Leanne Graham

email

Sincere@april.biz

iat: 발급 시각
exp: 만료 시각
단위: Unix timestamp (초)

그림 2. jwt.io의 실제 디코딩 화면. 화면을 합성하거나 내용을 수정하지 않았다.

유효시간 확인

```
iat = 1792853244
exp = 1792853844
exp - iat = 600초 = 10분
```

검증 구분: jwt.io에서는 Header와 Payload를 확인했다. 비밀키를 입력하지 않았으므로 이 화면만으로 서명 검증을 주장하지 않는다. 같은 토큰을 로컬 jwt.verify(token, process.env.JWT_SECRET)로 검증해 서명이 유효함을 확인했다.

발급 시각: 2026.10.24 23:47:24 KST / 만료 시각: 23:57:24 KST
캡처된 토큰은 보고서 열람 시 만료되어 있을 수 있으며, 서버에서 새로 발급받을 수 있다.

오류 처리와 검증 결과

실제 HTTP 요청과 자동 테스트를 구분해 결과를 확인했다.

01 실행 중인 서버에 보낸 요청

Bret	200	토큰 발급·서명 확인
unknown	404	사용자 없음
bret	404	대소문자 구분
username 누락	400	username 입력 필요
username: 123	400	문자열 입력 검증

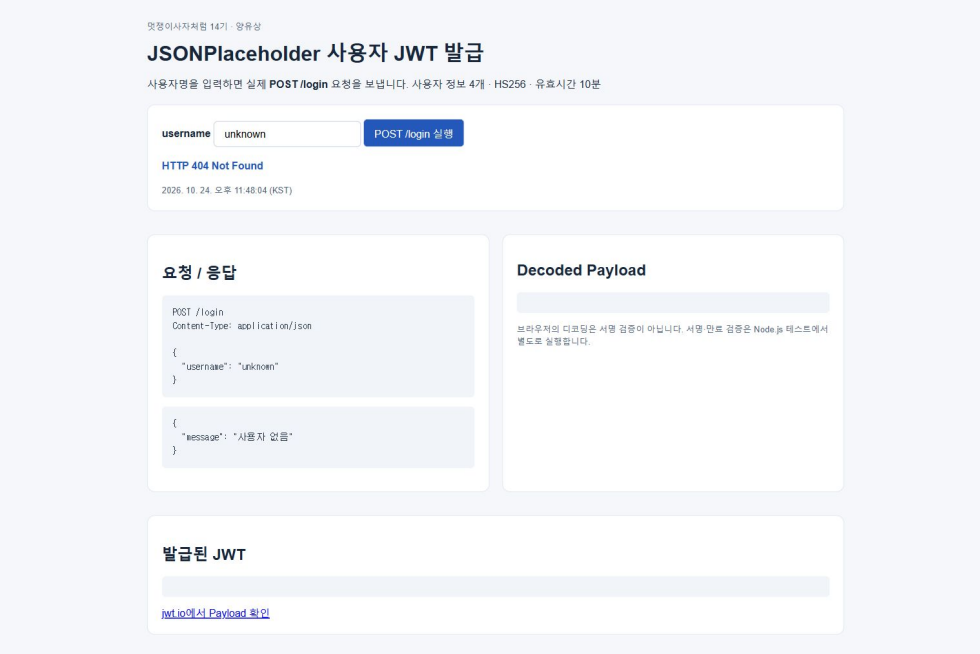


그림 3. unknown으로 실제 요청한 결과: HTTP 404, 사용자 없음.

02 자동 테스트: 8개 통과 / 실패 0개

정상 서명과 Payload, 600초 만료, 잘못된 키 및 변조 토큰 거부를 검증했다. 사용자 미발견, 잘못된 입력, 비밀키 누락, 외부 API 오류, 잘못된 JSON 본문 처리도 테스트했다.

만료 경계는 검증 시계를 exp - 1과 exp로 지정해 확인했다. 실제로 10분을 기다린 시험은 아니다. 외부 API 실패는 모의 응답으로 재현했고, 정상 발급은 실제 외부 API로 별도 검증했다.

트러블슈팅과 정리

실제로 확인한 문제, 수정 사항, 재검증 결과를 기록했다.

01 정상 사용자도 404가 발생

증상: 수정 전 기본 코드에 Bret를 전달했지만 status 404와 "사용자 없음"이 반환됐다.
원인: 사용자 검색이 const user = undefined로 남아 있었다.
해결: users.find()로 username을 정확히 비교하도록 구현했다.
확인: 실제 POST /login이 HTTP 200을 반환하고 네 가지 사용자 정보가 토큰에 들어갔다.

02 npm 설치 후 의존성 취약점 경고

증상: 첫 npm install에서 high 등급 취약점 1건이 표시됐다.
원인: body-parser의 압축 요청 처리 관련 취약점이었다.
해결: npm audit fix로 body-parser 2.4.0을 반영하고 lockfile을 갱신했다.
확인: npm audit 결과 취약점 0건, 자동 테스트 8개 통과를 확인했다.

03 요청 본문이 없을 때의 오류 처리 보완

req.body.username의 직접 접근을 req.body?.username으로 변경했다. 입력 검증은 issueToken에서 수행하도록 유지했고, 빈 HTTP 본문과 잘못된 JSON을 실제 요청해 400 응답을 확인했다.

배운 점과 실습 범위

JWT 발급은 사용자 조회, 필요한 정보 선택, 서명, 만료시간 설정으로 이어진다. Payload 디코딩은 내용을 읽는 작업이고, 서명 검증은 토큰이 변경되지 않았는지 확인하는 별도 작업이다. 이 실습은 공개 사용자명의 존재만 확인하므로 실제 서비스의 비밀번호 인증을 대체하지 않는다.

자료와 재현 방법

저장소에서 npm ci → .env 설정 → npm run dev 순서로 실행한다.
npm test로 자동 테스트, node scripts/verify-live.js로 실제 서버 요청을 재현한다. 원본 캡처와 검증 로그는 docs/evidence/에 보관했다.

JSONPlaceholder 사용자 API · jwt.io 디버거 · body-parser 보안 권고